

WEEK3 - TASK2

PCA

Principal Component Analysis

목차

01

PCA란?

PCA란 무엇이며, 왜 필요할까요?

02

PCA 작동 원리

PCA 작동 원리

03

과제 풀이

과제 풀이



PCA란 무엇이며, 왜 필요할까요?

PCA는 쉽게 우리가 분석해야 할 데이터가 너무 많은 경우가 있습니다. 예를 들어, 사진 한 장에는 수 천 개의 픽셀이 있지만, 사실 이 중 대부분은 분석에 크게 중요하지 않을 수 있습니다.

PCA는 데이터에서 정말 중요한 부분만 남기고 덜 중요한 부분을 제거하는 과정입니다.

장점

데이터 차원이 너무 많아 분석이 어렵거나 부정확해지는
차원의 저주를 완화할 수 있음
데이터 시각화가 용이해짐
계산 속도 및 효율성을 높일 수 있음



단점

주성분은 원래 **데이터의 의미를 잃을 수** 있어,
중요한 정보를 충분히 보존하지 못할 가능성이 있음



PCA 작동 원리

PCA는 주성분 분석의 약자로, 크게 주성분 (Principal Component)을 찾는 과정이 주 목표라고 할 수 있습니다.

01 데이터 표준화(정규화)

모든 데이터를 같은 크기(스케일)을 맞춰줍니다.
평균이 0, 표준편차가 1이 되도록 변환합니다.

02 분산 계산

데이터가 얼마나 흩어져있는지(분산)을 계산합니다.
이는 데이터의 각 특성 간 공분산(서로의 관계)을 통해 계산할 수 있습니다.

03 주성분(Principal Components) 찾기

데이터의 분산을 가장 잘 설명하는 방향(축)인 주성분을 찾는 과정입니다.
공분산 행렬의 고유값(각 주성분의 중요성)과 고유벡터(각 주성분의 방향)을 계산합니다.
- 첫번째 주성분은 데이터의 가장 큰 분산을 설명하고,
두번째 주성분은 첫번째 주성분에 수직이면서 그 다음으로 큰 분산을 설명합니다.

04 데이터 변환

데이터를 주성분 방향으로 변환합니다. 이 과정에서 데이터의 차원이 줄어들게 됩니다.



PCA 작동 원리

현재 키와 몸무게 데이터(2차원)를 통해,
체격(1차원)이라는 축으로 축소

- 첫번째 주성분이 95%의 분산을 설명한다면,
차원 축소시에 95%의 정보를 유지
- 두번째 주성분은 5%의 남은 정보를 설명하는 축으로,
분산이 적어 제거가 됨

01 데이터 표준화(정규화)

현재 키(x축)는 cm, 몸무게(y축)는 kg으로 크기와 단위가 다름

각 값에서 평균을 빼고, 표준편차로 나눔 -> 평균이 0, 표준편차가 1인 데이터로 변환

스케일이 다른 숫자가 큰 키의 영향이
몸무게보다 클 수 있음

02 분산 계산

키가 클수록 몸무게가 크다

키와 몸무게가 양의 상관관계를 가진다면, 값은 양수입니다.
값이 클수록, 두 특성이 서로 밀접하게 관련되어 있습니다.

$$\text{Cov} = \begin{bmatrix} \text{Var}(\text{키}) & \text{Cov}(\text{키}, \text{몸무게}) \\ \text{Cov}(\text{키}, \text{몸무게}) & \text{Var}(\text{몸무게}) \end{bmatrix}$$

03 주성분(Principal Components) 찾기

각 축이 데이터의 분산을 얼마나 설명하는지(고유값) * 클수록, 잘 설명
, 데이터가 가장 많이 흩어진 방향(고유벡터)를 계산
=> 고유값이 가장 큰 축=첫번째 고유벡터=첫번째 주성분
=> 두번째로 큰 고유값에 해당하는 고유벡터=두번째 주성분

04 데이터 변환

기존의 키, 몸무게 축을 제거하고, 새로운 축으로 데이터를 표현합니다.

과제 풀이



```
# 특성과 타겟 분리
X = data_encoded.drop('class', axis=1) if 'class' in data_encoded.columns else data_encoded.iloc[:, :-1]
y = data_encoded['class'] if 'class' in data_encoded.columns else data_encoded.iloc[:, -1]

# 데이터 정규화
scaler = StandardScaler()
X_normalized = scaler.fit_transform(X)
```

X(특성) : 데이터에서 입력값(설명 변수)을 추출

Y(타겟) : 분류하고자 하는 결과값(타겟 변수)을 추출

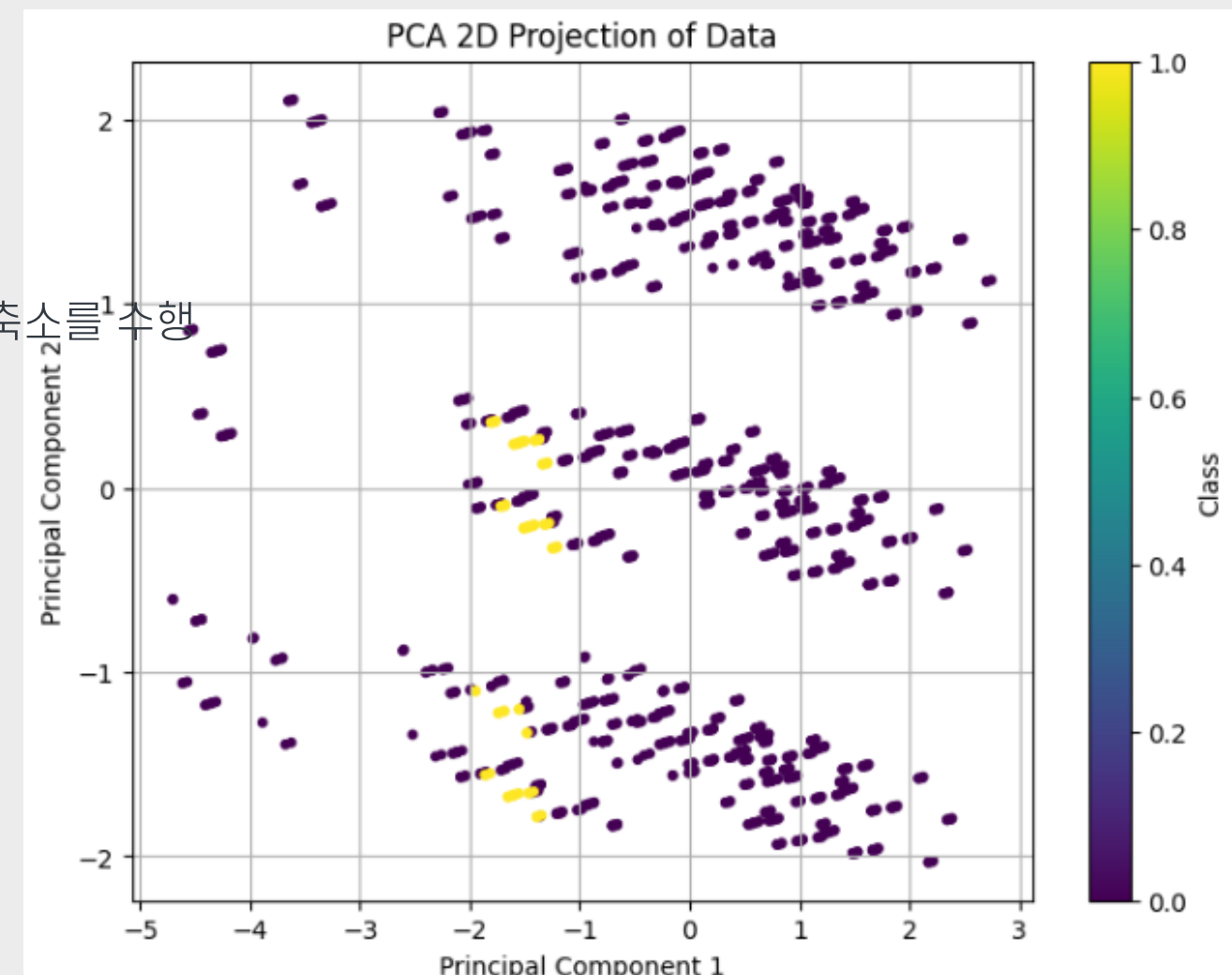
각 특성의 평균을 0, 표준편차를 1로 변환하여 데이터 스케일을 맞춥니다

```
# PCA 적용
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_normalized)

# 데이터 분포 시각화
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', s=10)
plt.colorbar(label='Class')
plt.title('PCA 2D Projection of Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid()
plt.show()
```

PCA 클래스에서 주성분을 2개로 지정하여 2차원으로 차원 축소를 수행

X_pca : 2개의 주성분으로 구성된 데이터



과제 풀이



데이터 분할

```
X_train, X_test, y_train, y_test = train_test_split(X_normalized, y, test_size=0.2, random_state=42)
X_pca_train, X_pca_test, _, _ = train_test_split(X_pca, y, test_size=0.2, random_state=42)
```

PCA 적용 전 성능 평가

```
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
rf_accuracy_before = accuracy_score(y_test, rf_model.predict(X_test))
```

PCA 적용 후 성능 평가

```
rf_model_pca = RandomForestClassifier(random_state=42)
rf_model_pca.fit(X_pca_train, y_train)
rf_accuracy_after = accuracy_score(y_test, rf_model_pca.predict(X_pca_test))
```

```
print(f"Random Forest Accuracy (Before PCA): {rf_accuracy_before:.4f}")
```

```
print(f"Random Forest Accuracy (After PCA): {rf_accuracy_after:.4f}")
```

80%는 훈련용, 20%는 테스트용으로 분할
PCA를 적용한 데이터, 아닌 데이터로 분할

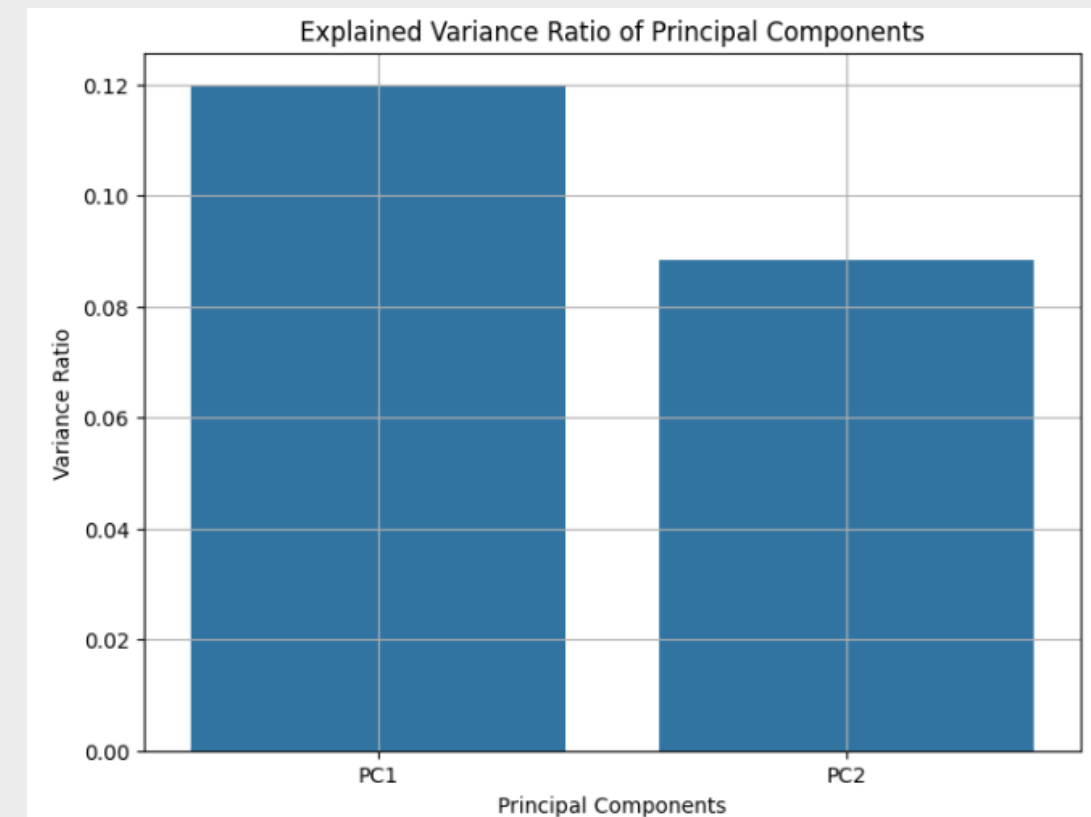
Random Forest Accuracy (Before PCA): 0.9827

Random Forest Accuracy (After PCA): 0.9884

과제 풀이

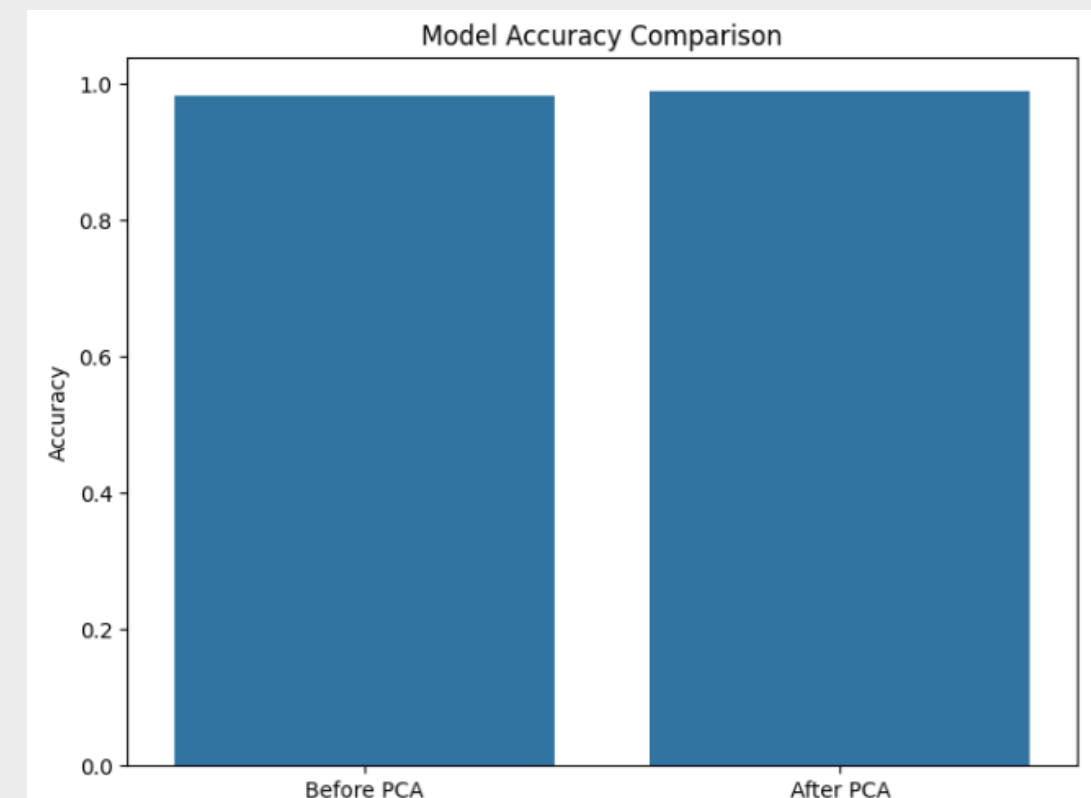
각 주성분의 주성분 분산 비율 시각화

```
# PCA 축소 결과 시각화
explained_variance_ratio = pca.explained_variance_ratio_
plt.figure(figsize=(8, 6))
sns.barplot(x=[f'PC{i+1}' for i in range(len(explained_variance_ratio))], y=explained_variance_ratio)
plt.ylabel('Variance Ratio')
plt.xlabel('Principal Components')
plt.grid()
plt.show()
```



앞의 PCA 적용 전후 랜덤포레스트 모델의 성능 차이를 시각화

```
# 모델 성능 비교 시각화
plt.figure(figsize=(8, 6))
models = ['Before PCA', 'After PCA']
accuracies = [rf_accuracy_before, rf_accuracy_after]
sns.barplot(x=models, y=accuracies)
plt.title('Model Accuracy Comparison')
plt.ylabel('Accuracy')
plt.show()
```





Thank You

감사합니다.